Curso: Ciência da Computação
Disciplina de Computação Gráfica

Aula 06
Primitivas Geométricas - Círculos

Professor: André Flores dos Santos

Santa Maria – RS 2026

SUMÁRIO

01 INTRODUÇÃO

02 CÍRCULOS

03 RASTERIZAÇÃO

04 ALGORITMOS

05 EXERCÍCIOS

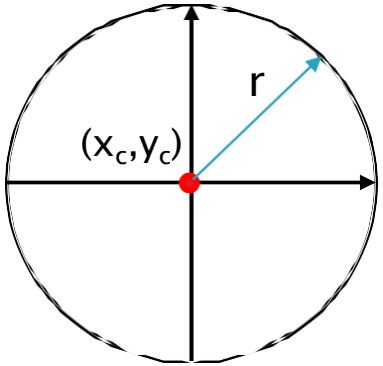
06

Círculos

- Círculos e arcos são fundamentais em Computação Gráfica, uma vez que permitem a geração de curvas e superfícies;
- O traçado de um círculo é definido por um conjunto de pontos que estão a uma mesma distância de um ponto:
  - Tal distância é o raio e o ponto é o centro do círculo;
- Matematicamente, a equação do círculo é dada por:

$$(x - x_c)^2 + (y - y_c)^2 = r^2$$

Círculos



- É útil também isolarmos a equação em termos de x e y:

$$x = x_c \pm \sqrt{r^2 - (y - y_c)^2}$$
$$y = y_c \pm \sqrt{r^2 - (x - x_c)^2}$$

- O "$\pm$" significa que, por exemplo, na 2ª expressão, para cada valor de x, são obtidos dois valores de y;

  Isso ocorre porque um círculo é simétrico em relação ao seu centro, portanto, existem dois pontos simétricos em relação ao eixo x (ou y) para cada x (ou y) específico.

Círculo na origem

- Para um círculo cujo ponto central é na **origem** ($x_c = 0$ e $y_c = 0$), a equação fica:

$$x^2 + y^2 = r^2$$

- Em Comp. Gráfica é interessante trabalharmos com **círculos na origem**, pelo fato de sua equação ser mais simples, e então aplicarmos transformações ao mesmo para modificar sua posição.

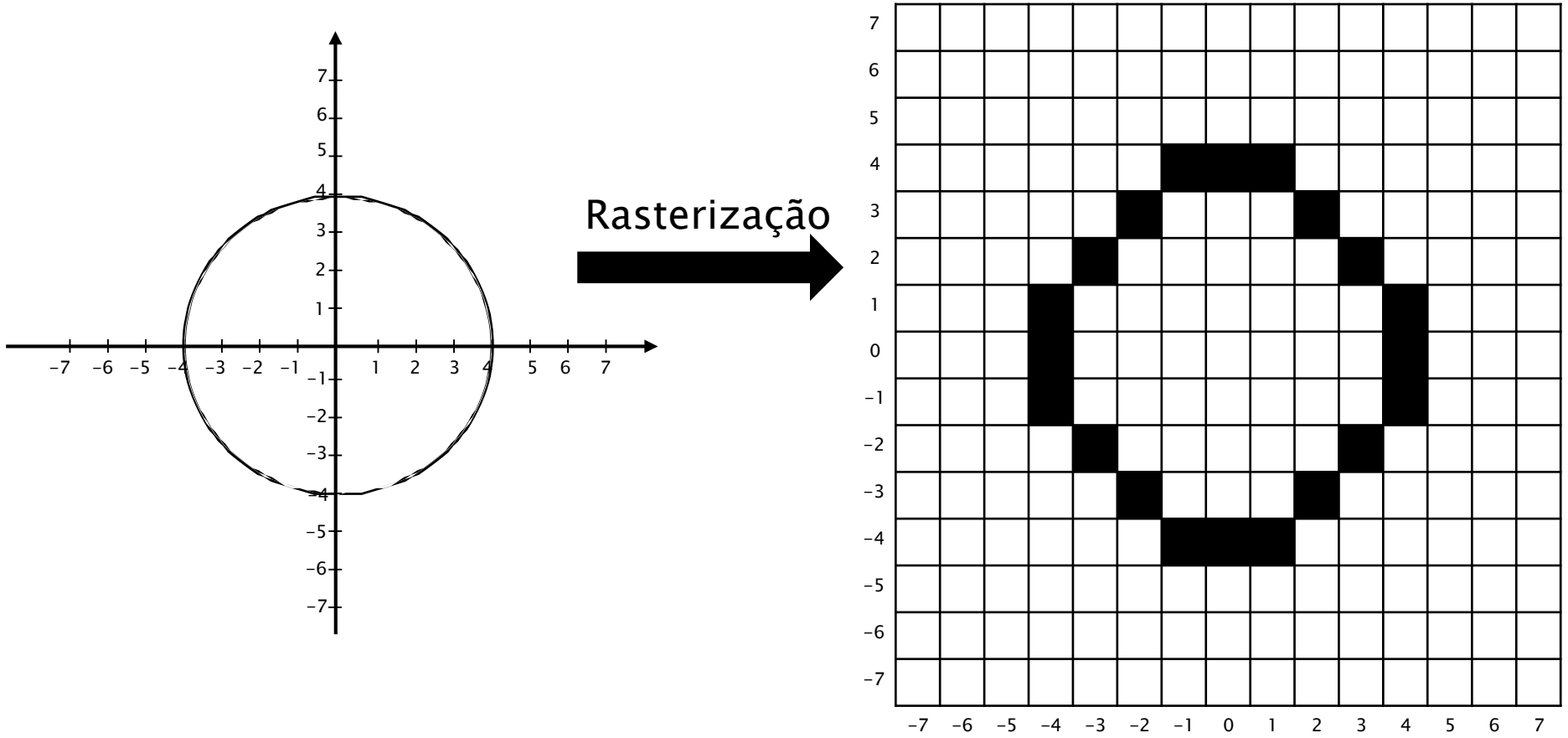
- Em termos de x e y, a equação fica:

$$x = \pm\sqrt{r^2 - y^2}$$

$$y = \pm\sqrt{r^2 - x^2}$$

Rasterização de círculos

- Problema:

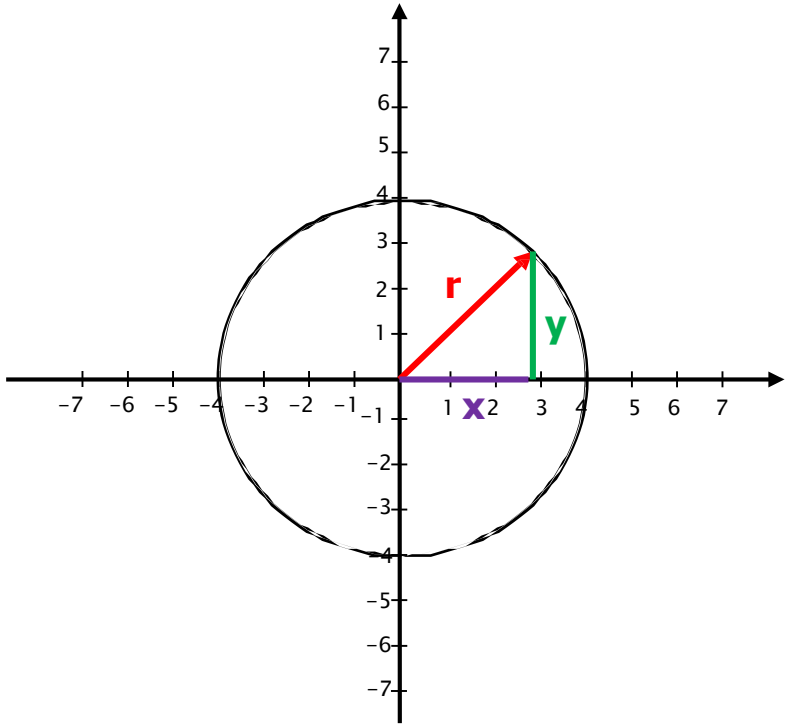


Solução 01 - Método da Equação Cartesiana

- Como vimos, a equação do círculo é:

$$x^2 + y^2 = r^2$$

- As variáveis da equação refletem o que é apresentado na figura abaixo:

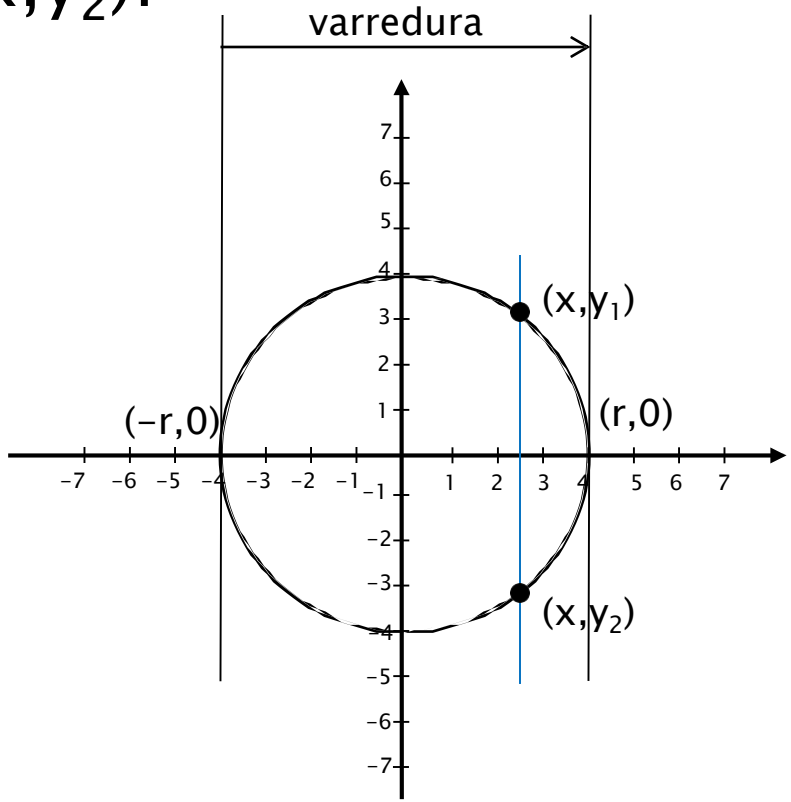


Solução 01

- Esse tipo de equação é denominado **equação implícita**, pois podemos substituir nela os valores para **x**, **y** e **r** para verificar se a mesma permanece verdadeira.
  - Caso positivo, isso significa que o ponto (x,y) pertence ao círculo.
  - Ex: Em um circunferência de raio = 4, o ponto (4,0) pertence ao circunferência?
    - $x^2 + y^2 = r^2$
    - $4^2 + 0^2 = 4^2$
    - $\mathbf{16 = 16}$
    - Portanto, o ponto (4,0) pertence a circunferência.

Solução 01

- Na figura abaixo, podemos perceber que para cada **x**, que varia de **−r** até **r**, temos dois valores possíveis para **y**, que são os pontos (x,y_1) e (x,y_2):



Solução 01

- Portanto, a solução mais simples seria fazer uma varredura para cada **x** de **−r** até **r**, para descobrirmos os valores de y_1 e y_2. Desta forma, ao final da varredura, nós poderemos desenhar um círculo;

- Sendo assim, precisamos determinar os valores de y_1 e y_2 para cada **x**;

- Utilizando a equação implícita da circunferência, rearranjamos os termos da mesma de modo a isolar o y:

$$x^2 + y^2 = r^2$$
$$y^2 = r^2 - x^2$$
$$\boldsymbol{y = \pm\sqrt{r^2 - x^2}}$$

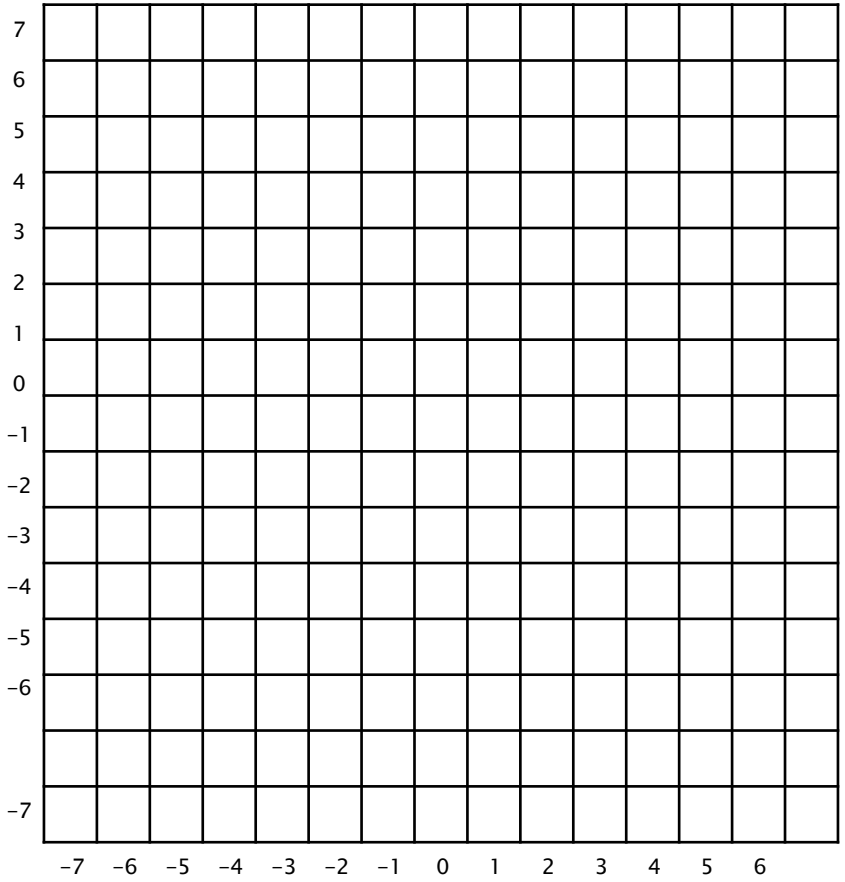
- A raiz quadrada, portanto, retorna um valor positivo e um valor negativo para **y**, ou seja, y_1 e y_2.

Solução 1

- Exemplo: rasterizar um círculo de raio = 4

x									
y_1									
y_2									

Vamos calcular, para cada **x** de **–4** até **4**, os valores de $\mathbf{y_1}$ e $\mathbf{y_2}$



Solução 1

- Exemplo: rasterizar um círculo de raio = 4

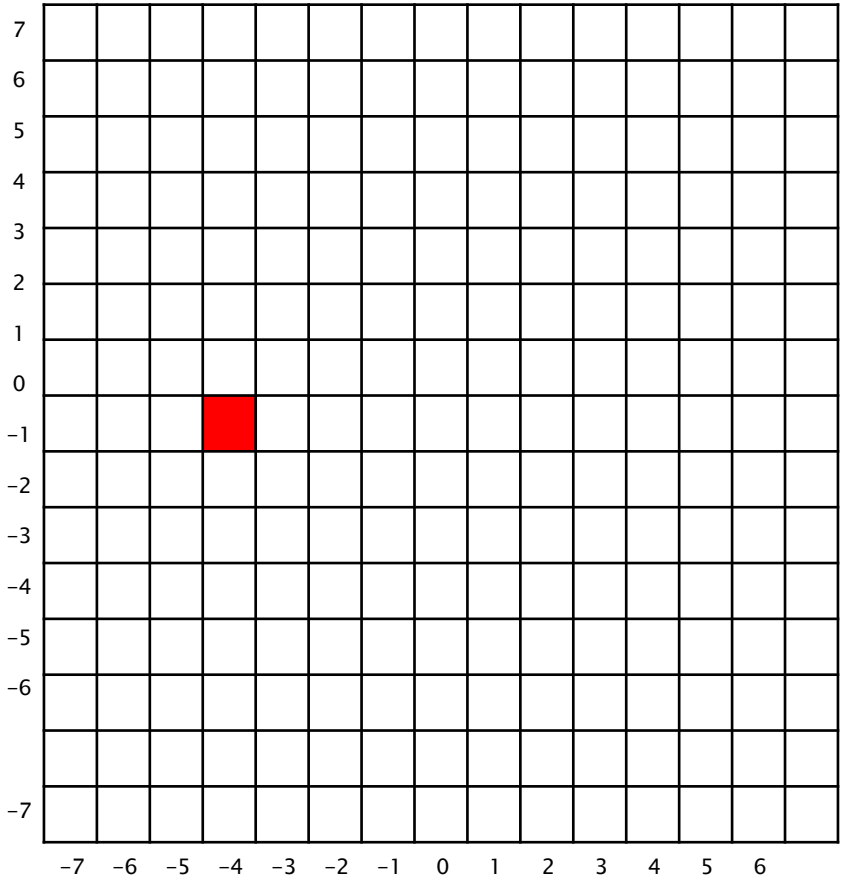
x	-4								
y_1	0								
y_2	0								

Para x=−4:

$$\boldsymbol{y = \sqrt{r^2 - x^2}}$$
$$\boldsymbol{y = \sqrt{4^2 - (-4^2)}}$$
$$\boldsymbol{y = \sqrt{16 - 16}}$$
$$\boldsymbol{y = 0}$$

→ $y_1=0$
→ $y_2=0$

Pintamos os pontos **(-4,0)** e **(-4,0)**



Solução 1

- Exemplo: rasterizar um círculo de raio = 4

x	-4	-3							
y_1	0	3							
y_2	0	−3							

Para x=−3:

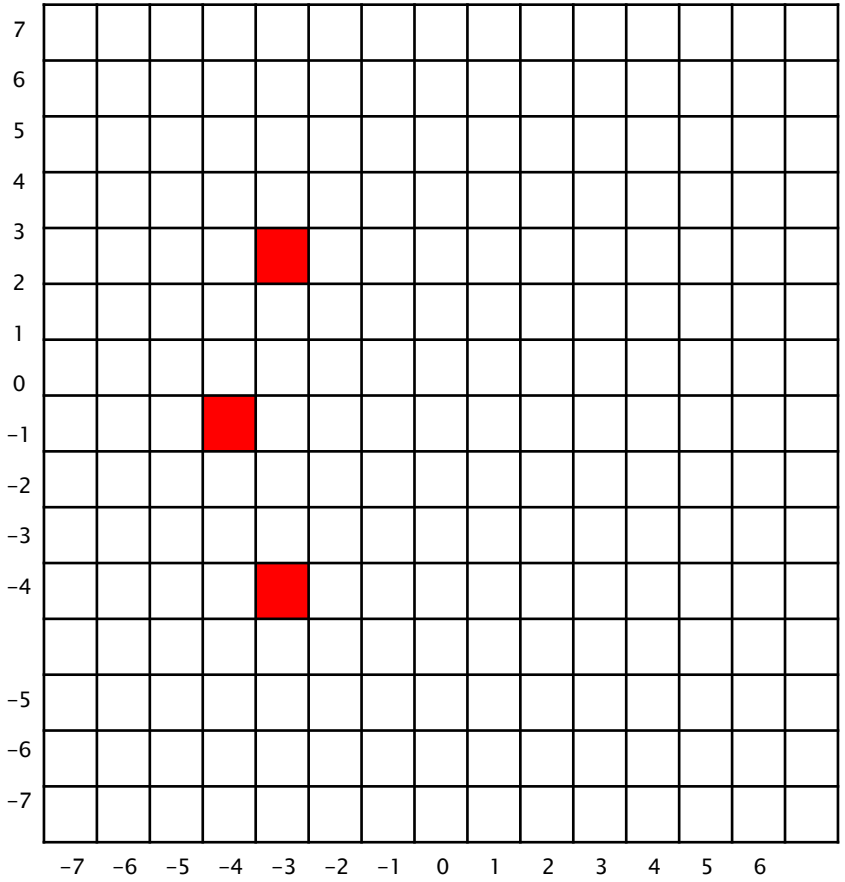
$$y = \sqrt{r^2 - x^2}$$
$$y = \sqrt{4^2 - (-3^2)}$$
$$y = \sqrt{16 - 9}$$
$$y = \sqrt{7} = 2.64$$

(arredondando)

→ $y_1 = 3$

→ $y_2 = -3$

Pintamos os pontos **(-3,3)** e **(-3,-3)**



Solução 1

- Exemplo: rasterizar um círculo de raio = 4

x	-4	-3	-2						
y_1	0	3	3						
y_2	0	−3	−3						

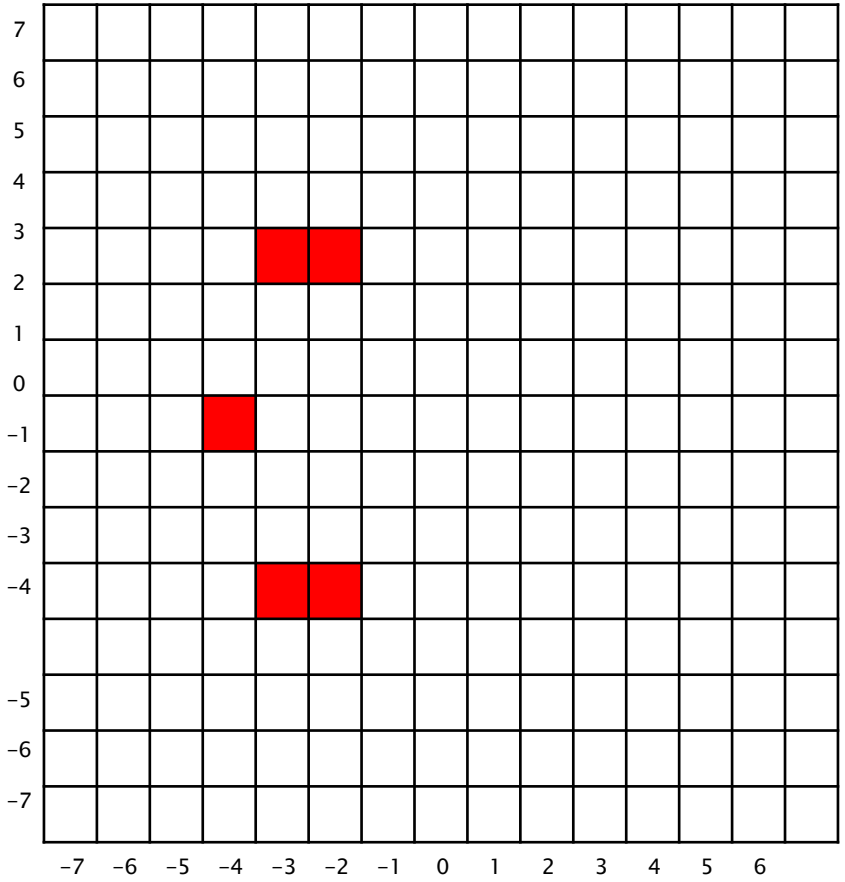
Para x=−2:

$$y = \sqrt{r^2 - x^2}$$
$$y = \sqrt{4^2 - (-2^2)}$$
$$y = \sqrt{16 - 4}$$
$$y = \sqrt{12} = 3.46$$

→ $y_1=3$

(arredondando) → $y_2=-3$

Pintamos os pontos **(-2,3)** e **(-2,-3)**



Solução 1

- Exemplo: rasterizar um círculo de raio = 4

x	-4	-3	-2	-1					
y_1	0	3	3	4					
y_2	0	−3	−3	−4					

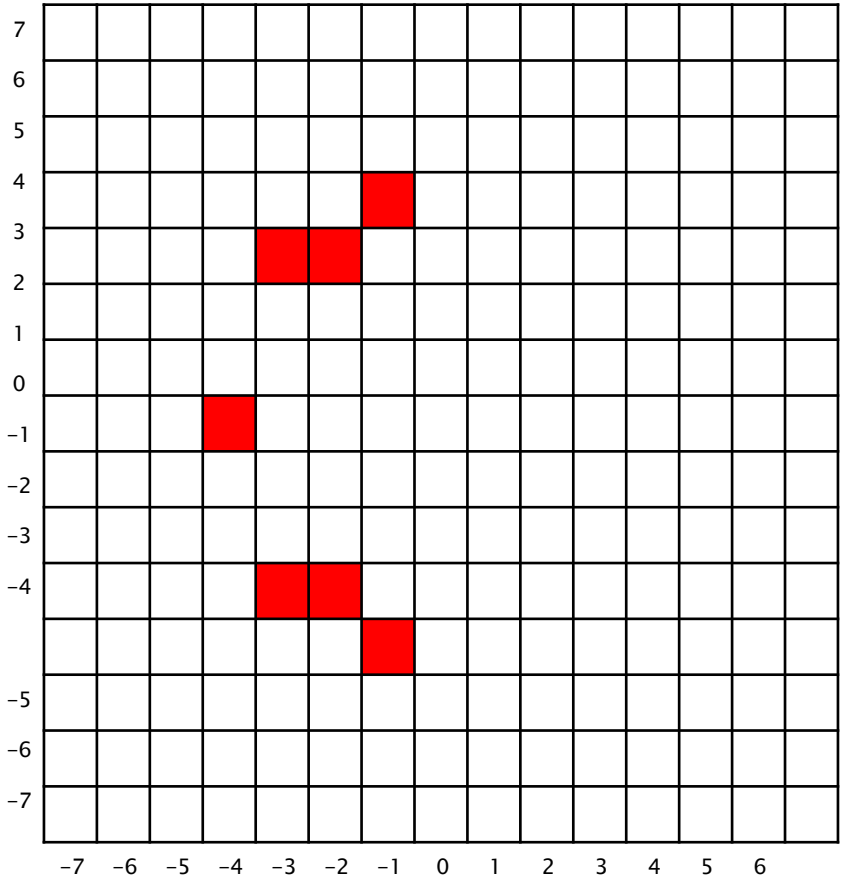
Para x=−1:

$$y = \sqrt{r^2 - x^2}$$
$$y = \sqrt{4^2 - (-1^2)}$$
$$y = \sqrt{16 - 1}$$
$$y = \sqrt{15} = 3.87$$

→ $y_1=4$

(arredondando) → $y_2=-4$

Pintamos os pontos **(-1,4)** e **(-1,-4)**



Solução 1

- Exemplo: rasterizar um círculo de raio $= 4$

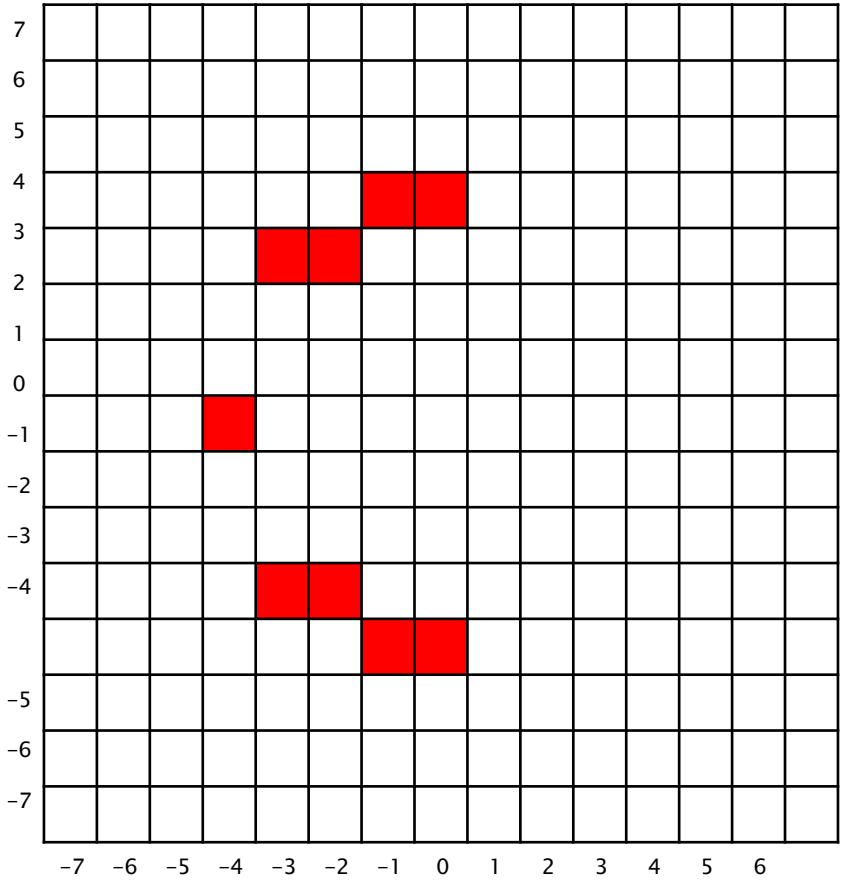
x	**-4**	**-3**	**-2**	**-1**	**0**				
y_1	0	3	3	4	4				
y_2	0	−3	−3	−4	−4				

Para x=0:

$$\boldsymbol{y = \sqrt{r^2 - x^2}}$$
$$\boldsymbol{y = \sqrt{4^2 - (-0^2)}}$$
$$\boldsymbol{y = \sqrt{16 - 0}}$$
$$\boldsymbol{y = \sqrt{16} = 4}$$

→ $y_1=4$
→ $y_2=-4$

Pintamos os pontos **(0,4)** e **(0,-4)**



Solução 1

- Exemplo: rasterizar um círculo de raio = 4

x	-4	-3	-2	-1	0	1			
y_1	0	3	3	4	4	4			
y_2	0	−3	−3	−4	−4	−4			

Para x=1:

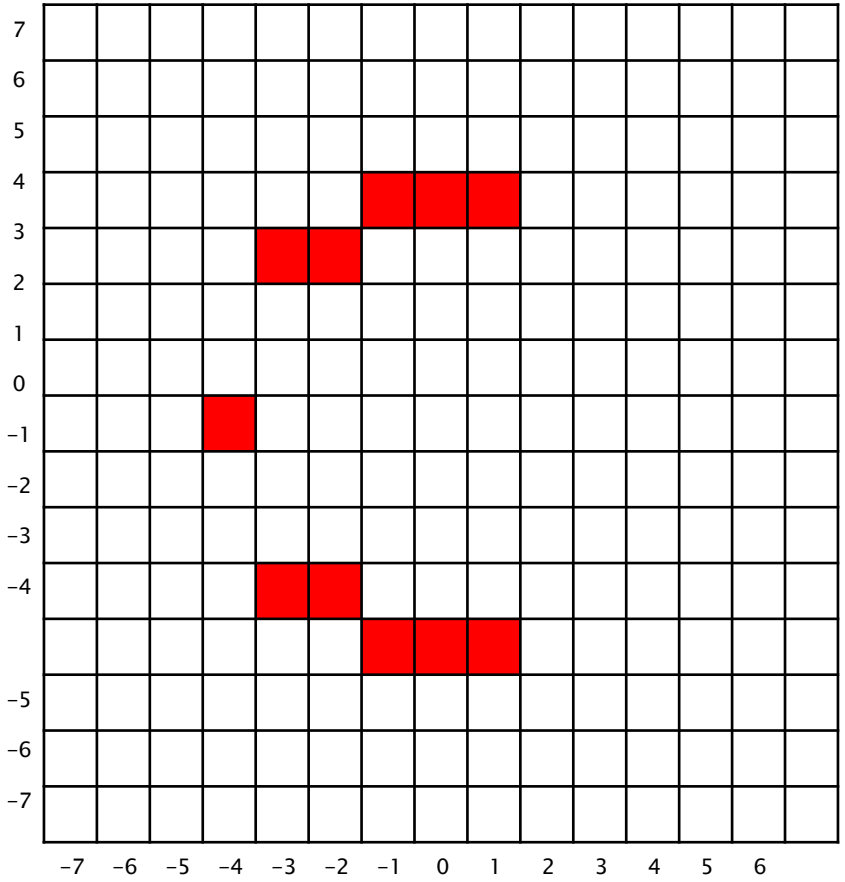
$$y = \sqrt{r^2 - x^2}$$
$$y = \sqrt{4^2 - (1^2)}$$
$$y = \sqrt{16 - 1}$$
$$y = \sqrt{15} = 3.87$$

(arredondando)

→ $y_1 = 4$

→ $y_2 = -4$

Pintamos os pontos **(1,4)** e **(1,-4)**



Solução 1

- Exemplo: rasterizar um círculo de raio = 4

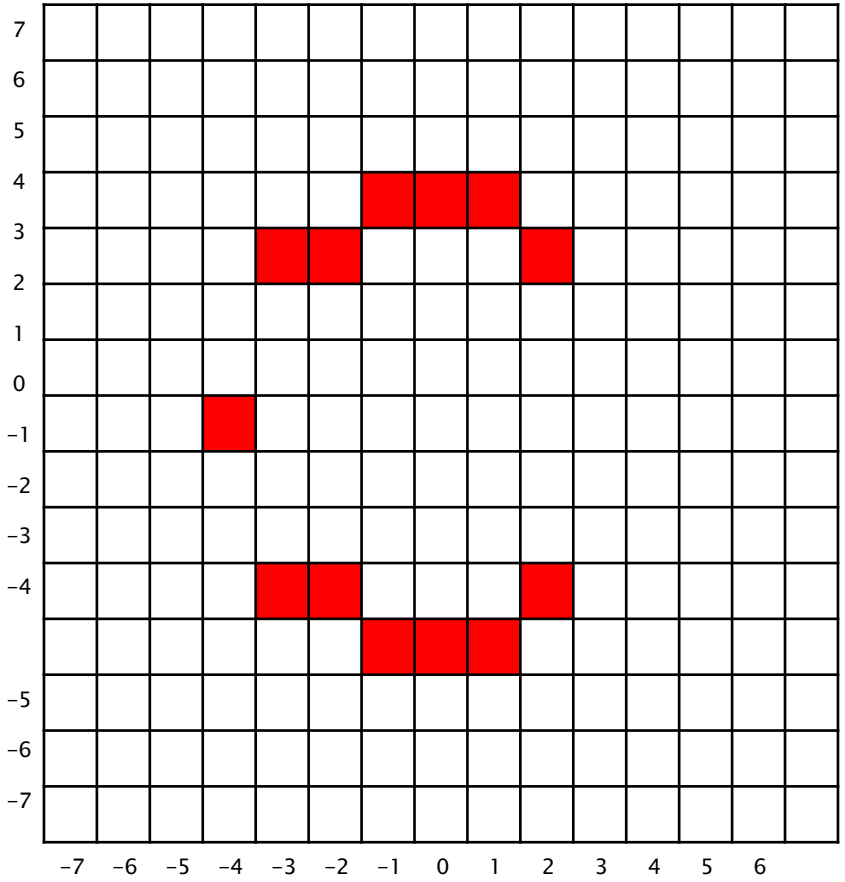
x	-4	-3	-2	-1	0	1	2		
y_1	0	3	3	4	4	4	3		
y_2	0	−3	−3	−4	−4	−4	−3		

Para x=2:

$$y = \sqrt{r^2 - x^2}$$
$$y = \sqrt{4^2 - (2^2)}$$
$$y = \sqrt{16 - 4}$$
$$y = \sqrt{12} = 3.46$$

(arredondando) → $y_1 = 3$
→ $y_2 = -3$

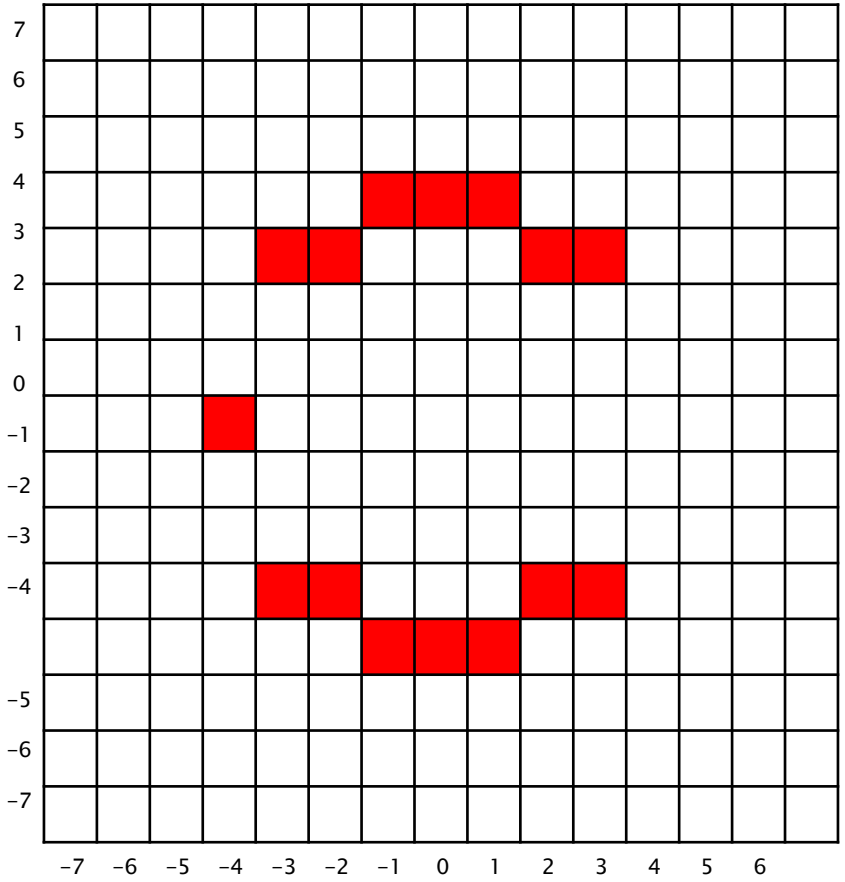
Pintamos os pontos **(2,3)** e **(2,-3)**



Solução 1

- Exemplo: rasterizar um círculo de raio = 4

x	-4	-3	-2	-1	0	1	2	3	
y_1	0	3	3	4	4	4	3	3	
y_2	0	−3	−3	−4	−4	−4	−3	−3	



Para x=3:

$$y = \sqrt{r^2 - x^2}$$
$$y = \sqrt{4^2 - (3^2)}$$
$$y = \sqrt{16 - 9}$$
$$y = \sqrt{7} = 2.64$$

(arredondando)

→ $y_1 = 3$

→ $y_2 = -3$

Pintamos os pontos **(3,3)** e **(2,-3)**

Solução 1

- Exemplo: rasterizar um círculo de raio = 4

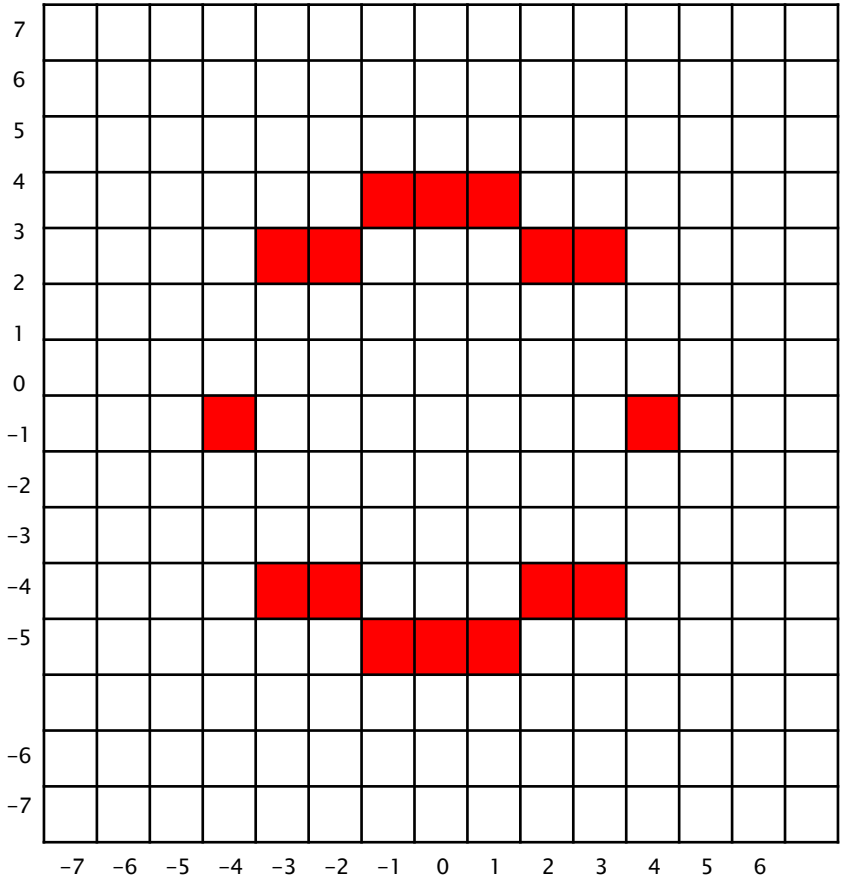
x	-4	-3	-2	-1	0	1	2	3	4
y_1	0	3	3	4	4	4	3	3	0
y_2	0	−3	−3	−4	−4	−4	−3	−3	0

Para x=4:

$$y = \sqrt{r^2 - x^2}$$
$$y = \sqrt{4^2 - (4^2)}$$
$$y = \sqrt{16 - 16}$$
$$y = 0$$

→ $y_1=0$
→ $y_2=0$

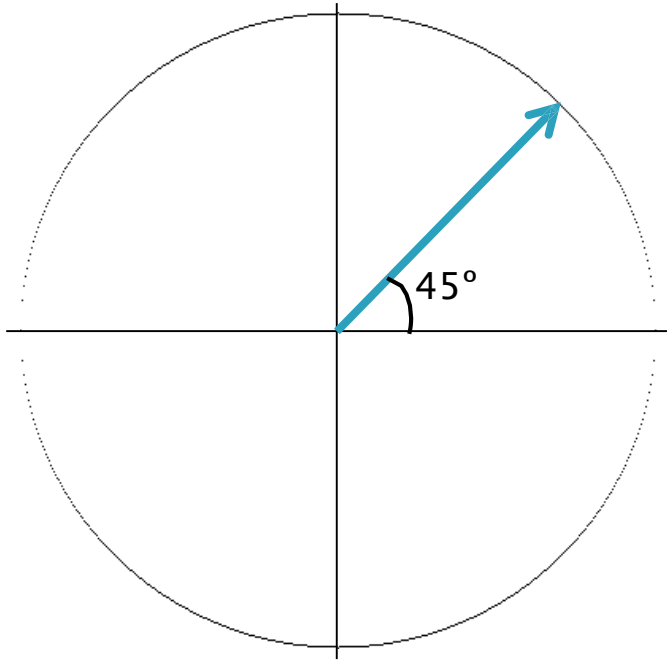
Pintamos os pontos **(4,0)** e **(4,0)**



Solução 1

- Como podemos perceber, a solução 1 rasteriza um círculo, porém o mesmo é **descontínuo**;
- Um detalhe importante de se observar é que a descontinuidade acontece entre os ângulos de 0 e 45°.
- O mesmo algoritmo, utilizado para rasterizar um círculo em uma resolução maior, resulta em algo parecido com isso:

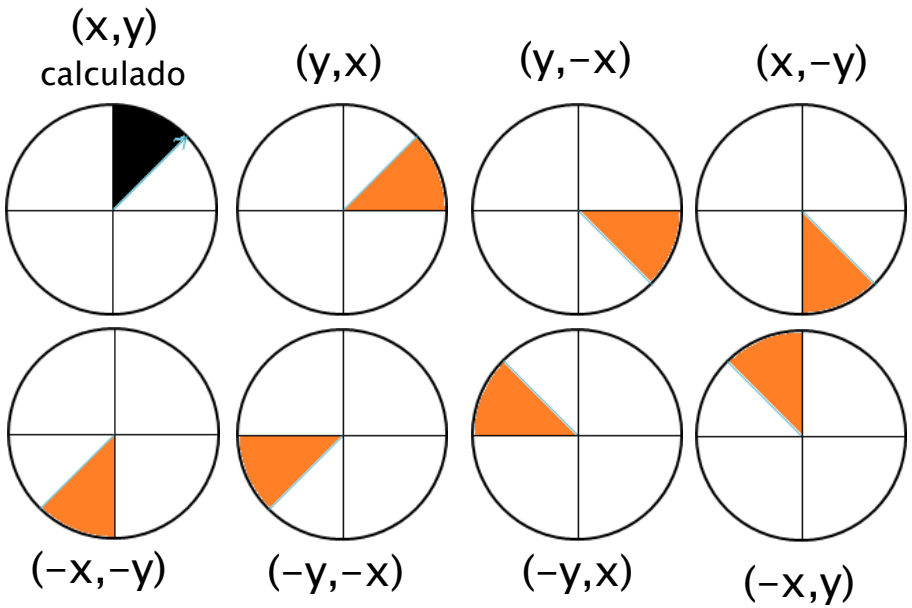
Observa-se que apesar da descontinuidade acontecer entre 0 e 45º, ela não acontece entre os ângulos de 45º e 90º!



Solução 02 - Método da Equação Paramétrica

- Na solução anterior, nós processamos somente um semicírculo:
  - Pois para cada x calculado, calculamos o valor de y correspondente (x,y) e então espelhamos o mesmo (x,–y);

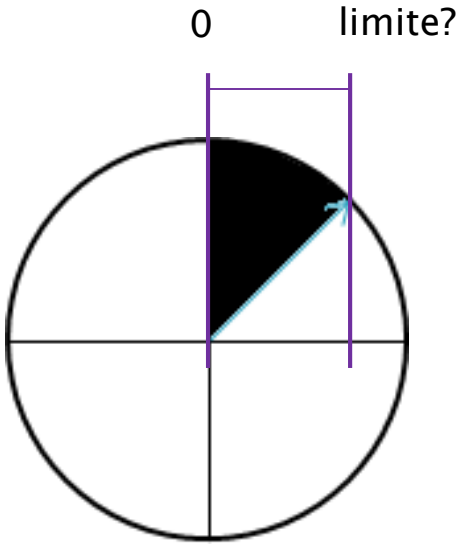
- Para resolver este problema, podemos seguir a mesma ideia, mas ao invés de calcular todos os pontos do semicírculo, calculamos somente os pontos entre **45º e 90º** (em que não há descontinuidade), e então espelhamos o mesmo nos **8 octantes**:



Solução 2

- Na solução 1, fizemos o **x** variar de **–r** até **r**, percorrendo valores de 0º a 180º (semicírculo);

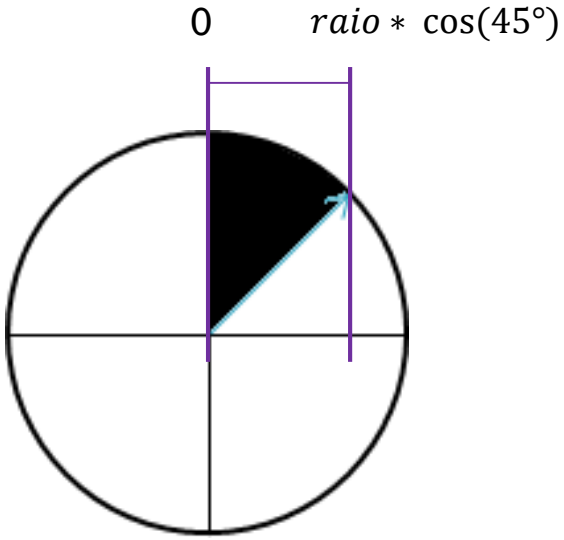
- Para percorrermos de 90º a 45º, precisamos fazer **x** variar de quanto até quanto?



Solução 2

- De acordo com as **coordenadas polares**, podemos definir que limite é:

$$limite = raio * \cos(45°)$$



Solução 2

- Exemplo: rasterizar um círculo de raio $= 4$

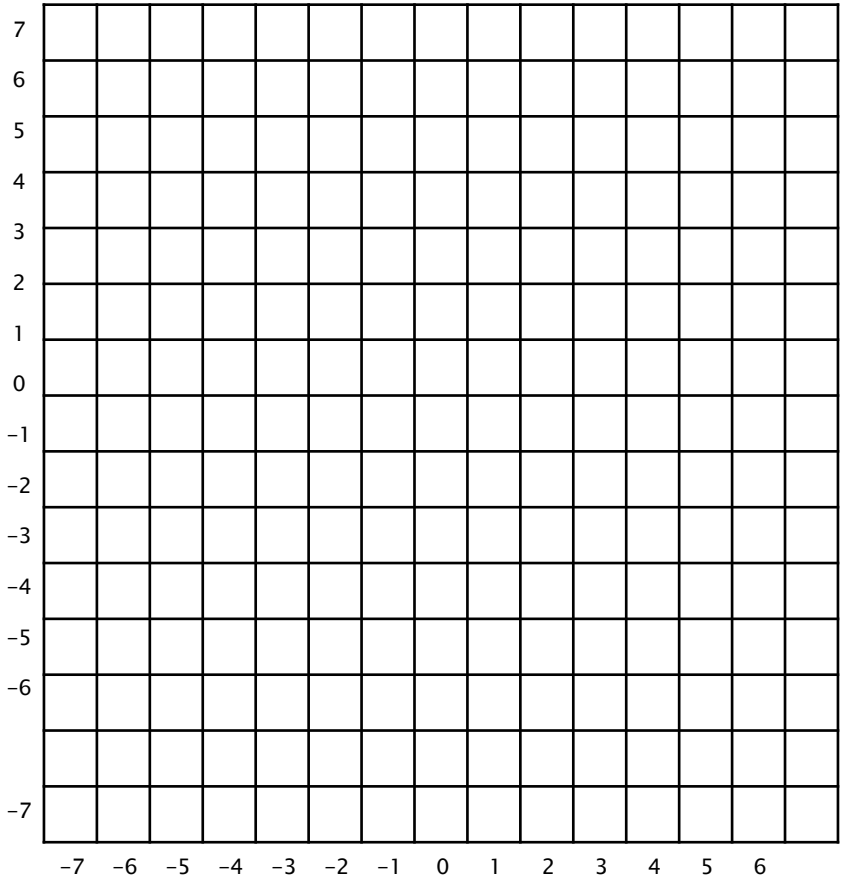
x			
y			

Calculamos o limite:

$limite = raio * cos(45°)$
$limite = 4 * cos(45°)$
$limite = 4 * 0.7071 = 2.828$

Como o limite precisa ser inteiro, podemos truncar para 2.

Desta forma, calculamos os valores de **y** para **x** variando de 0 a 2.



Solução 2

- Exemplo: rasterizar um círculo de raio = 4

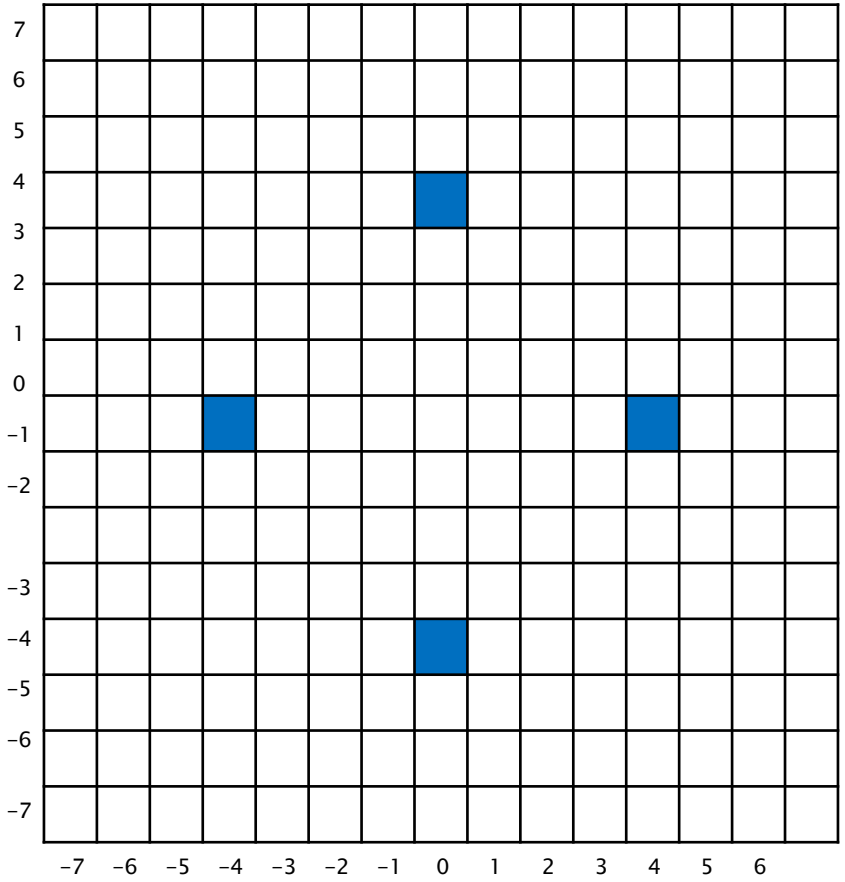
x	**0**		
y	4		

Para x=0:

$$\boldsymbol{y = \sqrt{r^2 - x^2}}$$
$$\boldsymbol{y = \sqrt{4^2 - (0^2)}}$$
$$\boldsymbol{y = 4}$$

Pintamos os pontos:

(x,y)	→ (0,4)	(y,x)	→ (4,0)
(x,−y)	→ (0,−4)	(−y,x)	→ (−4,0)
(−x,y)	→ (0,4)	(y,−x)	→ (4,0)
(−x,−y)	→ (0,−4)	(−y,−x)	→ (−4,0)



Solução 2

- Exemplo: rasterizar um círculo de raio = 4

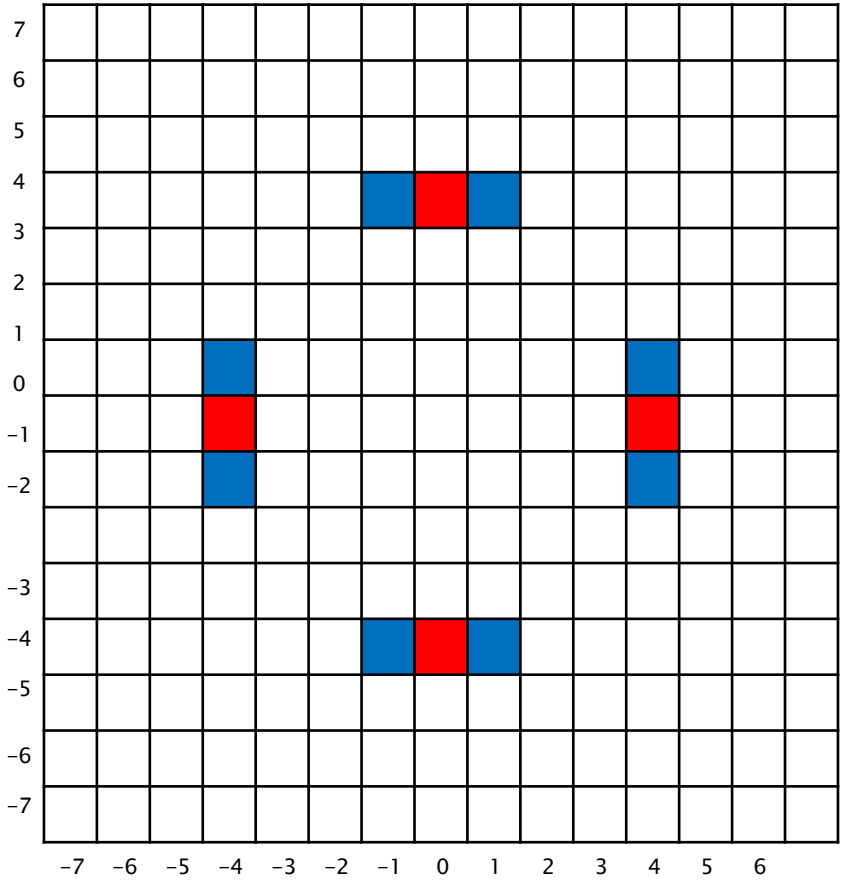
x	0	1	
y	4	4	

Para x=1:

$$y = \sqrt{r^2 - x^2}$$
$$y = \sqrt{4^2 - (1^2)}$$
$$y = 3.87 \cong 4$$

Pintamos os pontos:

(x,y)	→ (1,4)	(y,x)	→ (4,1)
(x,−y)	→ (1,−4)	(−y,x)	→ (−4,1)
(−x,y)	→ (−1,4)	(y,−x)	→ (4,−1)
(−x,−y)	→ (−1,−4)	(−y,−x)	→ (−4,−1)



Solução 2

- Exemplo: rasterizar um círculo de raio = 4

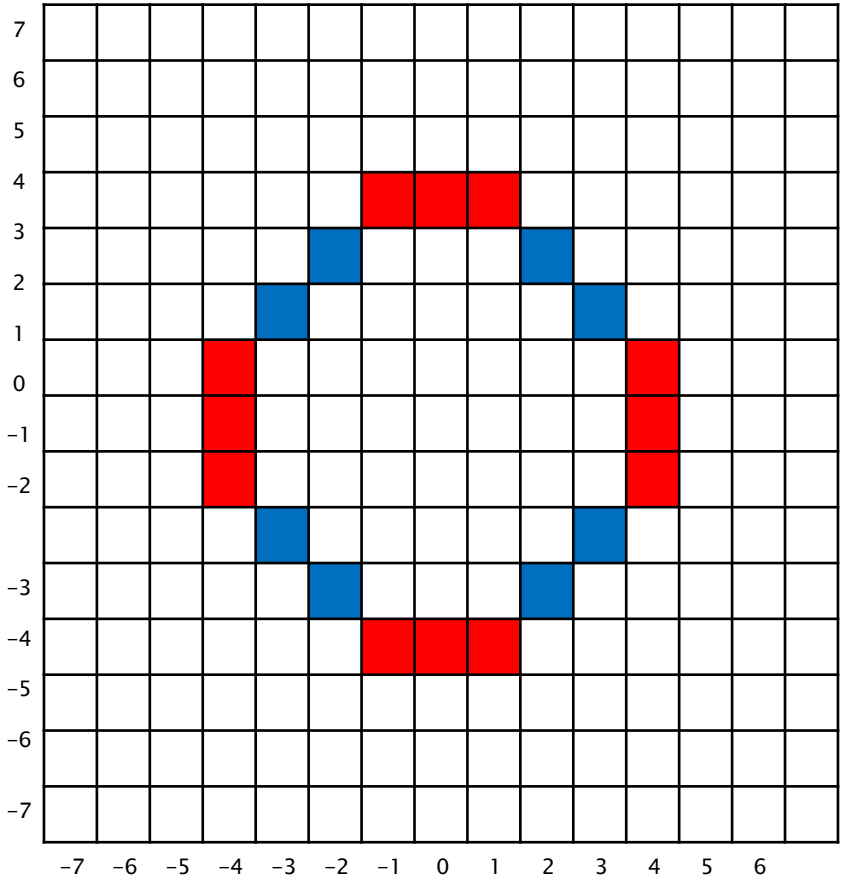
x	0	1	2
y	4	4	3

Para x=2:

$$y = \sqrt{r^2 - x^2}$$
$$y = \sqrt{4^2 - (2^2)}$$
$$y = 3.46 \cong 3$$

Pintamos os pontos:

(x,y)	→ (2,3)	(y,x)	→ (3,2)
(x,−y)	→ (2,−3)	(−y,x)	→ (−3,2)
(−x,y)	→ (−2,3)	(y,−x)	→ (3,−2)
(−x,−y)	→ (−2,−3)	(−y,−x)	→ (−3,−2)

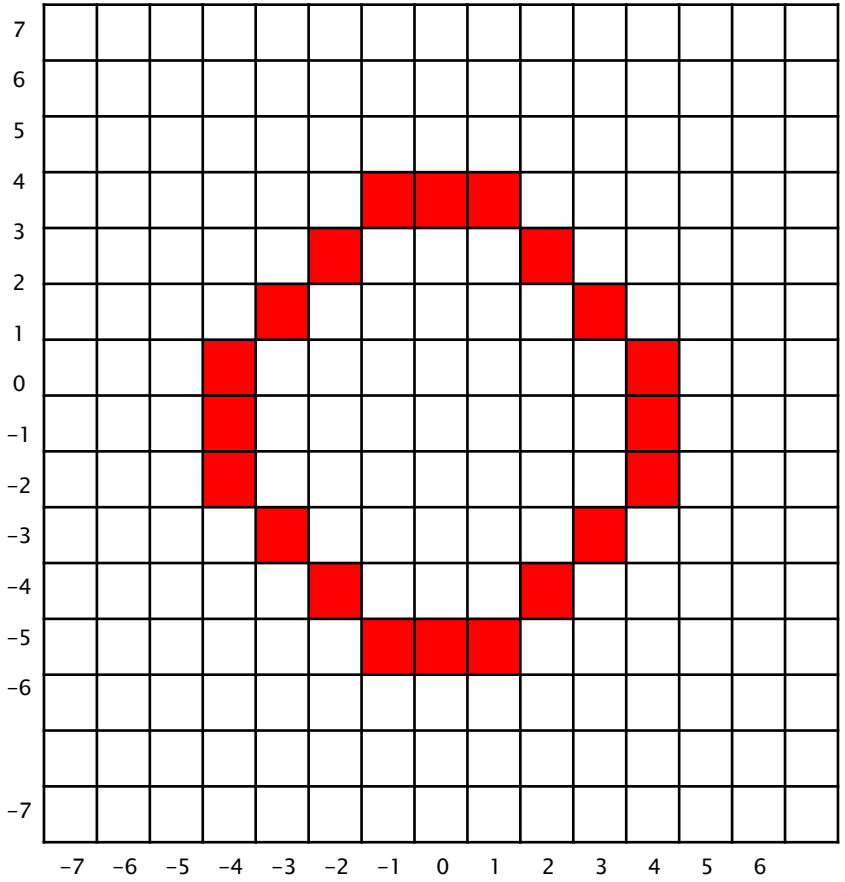


Solução 2

- Exemplo: rasterizar um círculo de raio = 4

x	0	1	2
y	4	4	3

Círculo finalizado!



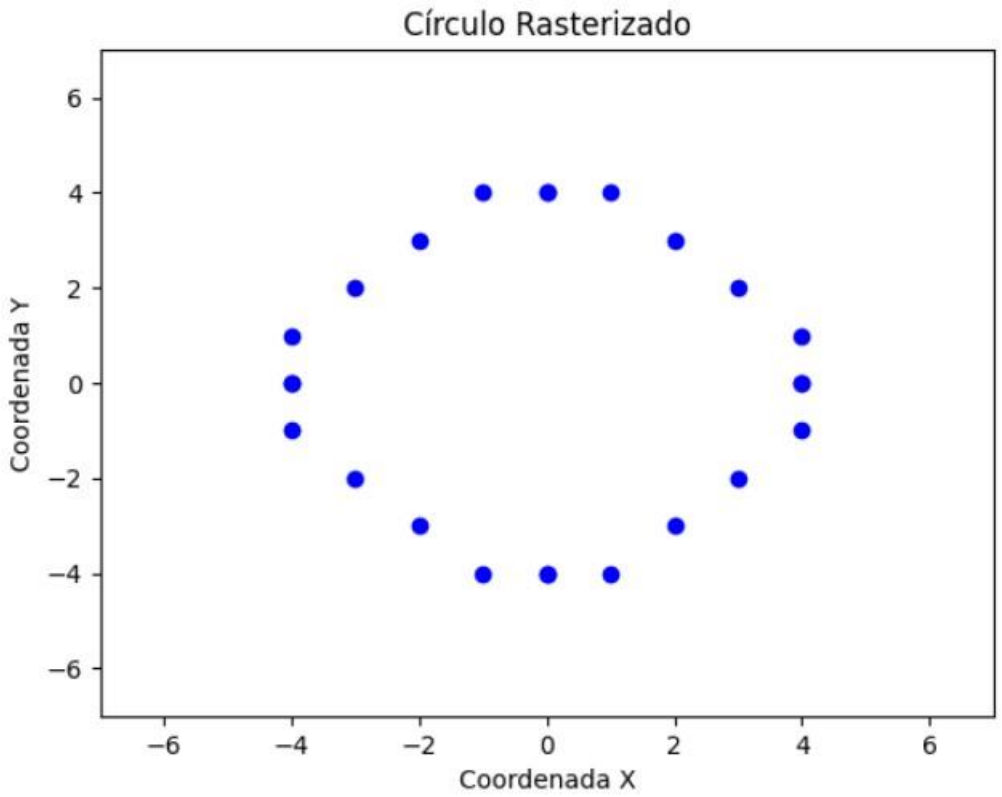
Solução 2 - Algoritmo em python

```python
import math
import matplotlib.pyplot as plt

# Defina o raio do círculo
raio = 4

# Inicialize a coordenada x
x = 0

# Lista para armazenar os pontos
pontos = []

# Calcule o limite para a varredura no eixo x
limite = raio * math.cos(math.radians(45))
print("\n Limite:",round(limite) )

# Loop enquanto x for menor que o limite
while x <= limite:
    print("\n X:",x)

    # Calcule a coordenada y correspondente usando a equação do círculo
    y = round(math.sqrt(raio * raio - x * x))
    print("(",x,",",y,")")
    print("(",y,",",x,")")
    print("(",y,",",-x,")")
    print("(",x,",",-y,")")
    print("(",-x,",",-y,")")
    print("(",-y,",",-x,")")
    print("(",-y,",",x,")")
    print("(",-x,",",y,")")

    # Adicione os pontos à lista considerando a simetria do círculo
    pontos.extend([(x, y), (y, x), (y, -x), (x, -y), (-x, -y), (-y, -x), (-y, x), (-x, y)])

    # Incremente x para avançar na varredura
    x += 1
```

```python
#Separe as coordenadas x e y para plotagem de forma simples
coordenadas_x = []
coordenadas_y = []


for p in pontos:
    coordenadas_x.append(p[0])  # O primeiro elemento da tupla é o X
    coordenadas_y.append(p[1])  # O segundo elemento da tupla é o Y


# Plote os pontos
plt.plot(coordenadas_x, coordenadas_y, 'bo’)
 # 'bo-' para pontos azuis conectados por linhas


# Adicione título e rótulos dos eixos
plt.title("Círculo Rasterizado")
plt.xlabel("Coordenada X")
plt.ylabel("Coordenada Y")


# Ajuste os limites dos eixos para -7 a 7
plt.xlim(-7, 7)
plt.ylim(-7, 7)


# Mostre o gráfico
plt.show()
```

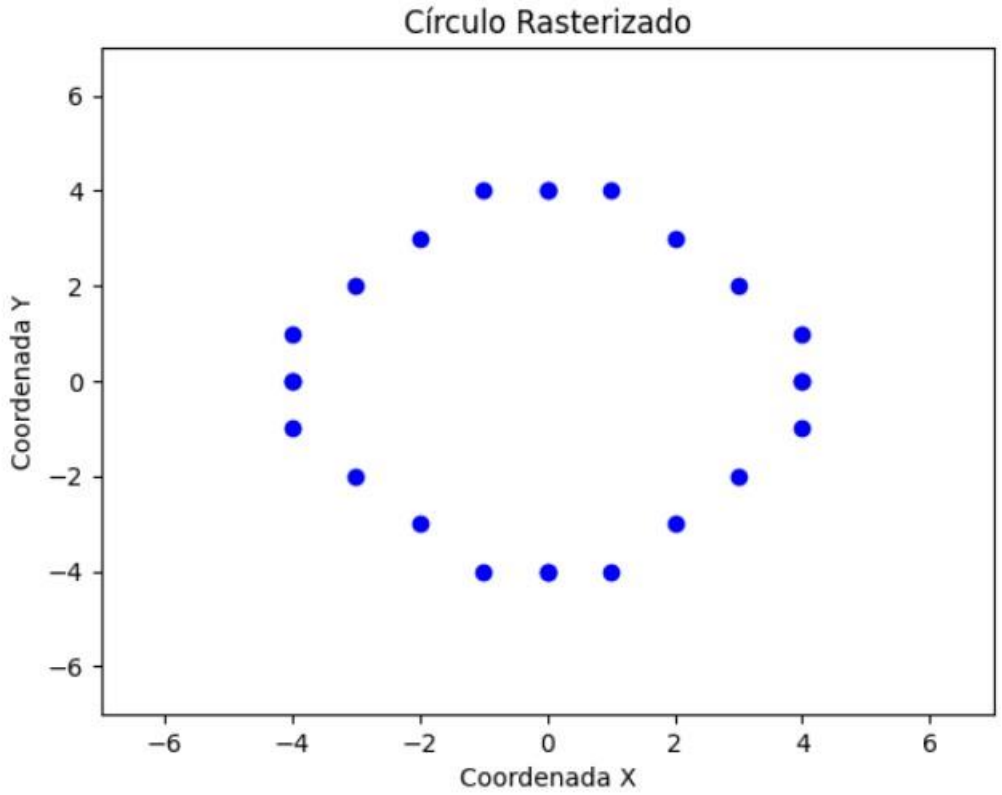
Solução 2 - Algoritmo em python

Resultado:



Solução 2 - Algoritmo em python

Se eu quiser deixar o círculo mais redondo? Teremos que adicionar mais valores para x, do tipo decimal.



Solução 2 - Algoritmo em python

Parte I do código (continua próximo slide)

Se eu quiser deixar o círculo mais redondo? teremos que adicionar mais valores para x, do tipo decimal. Ex: 0.1 a 0.1

```python
import math
import matplotlib.pyplot as plt

# Defina o raio do círculo
raio = 4

# Inicialize a coordenada x
x = 0.0  # Agora começa com um número decimal

# Lista para armazenar os pontos
pontos = []

# Calcule o limite para a varredura no eixo x
limite = raio * math.cos(math.radians(45))

# Loop enquanto x for menor que o limite
while x <= limite:
    # Calcule a coordenada y correspondente usando a equação do círculo
    y = round(math.sqrt(raio * raio - x * x), 2)  # Mantém valores decimais

    # Adicione os pontos à lista considerando a simetria do círculo
    pontos.extend([
        (round(x, 2), y), (y, round(x, 2)), (y, -round(x, 2)), (round(x, 2), -y),
        (-round(x, 2), -y), (-y, -round(x, 2)), (-y, round(x, 2)), (-round(x, 2), y)
    ])

    # Incremente x para avançar na varredura (agora em passos menores)
    x += 0.1  # Reduzindo o passo para melhorar a suavidade
```

Solução 2 - Algoritmo em python

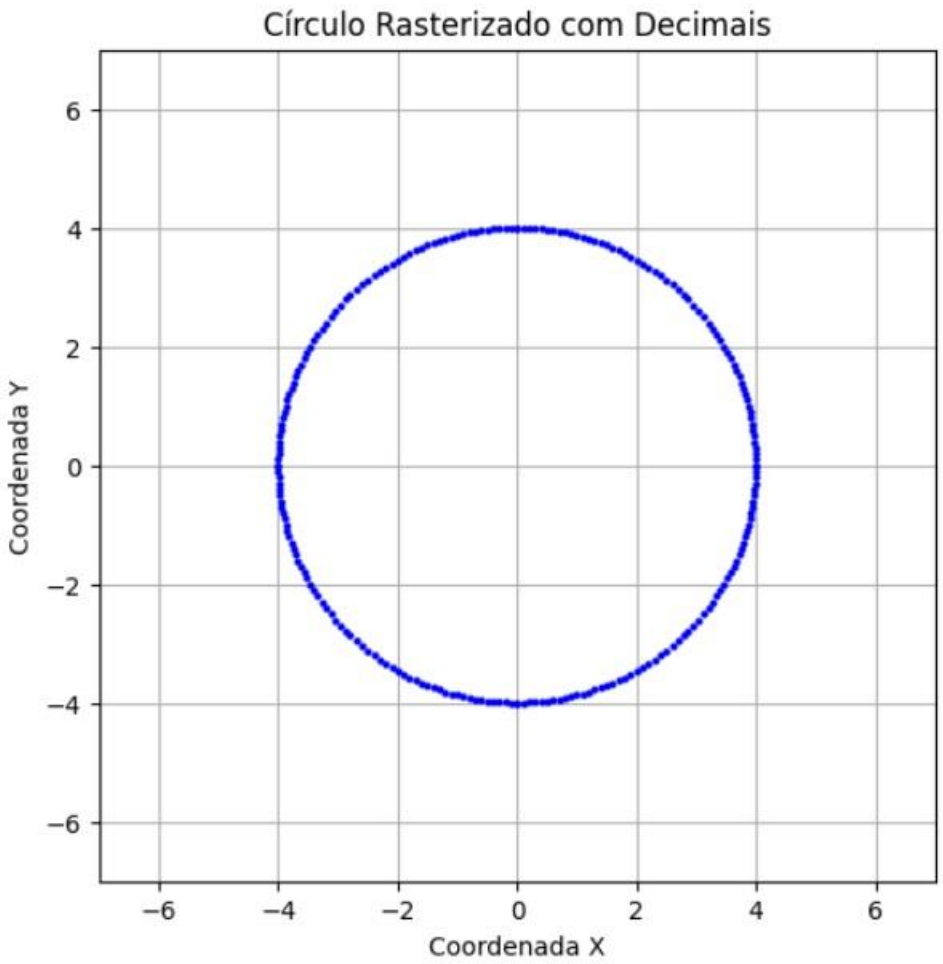
Parte II do código

Se eu quiser deixar o círculo mais redondo? teremos que adicionar mais valores para x, do tipo decimal. Ex: 0.1 a 0.1

```
# Separe as coordenadas x e y para plotagem
coordenadas_x = []
coordenadas_y = []

for p in pontos:
    coordenadas_x.append(p[0])  # O primeiro elemento da tupla é o X
    coordenadas_y.append(p[1])  # O segundo elemento da tupla é o Y

# Plote os pontos
plt.figure(figsize=(6,6))
plt.plot(coordenadas_x, coordenadas_y, 'bo', markersize=2)  # Pontos menores para
melhor visualização

# Ajuste os limites dos eixos para -7 a 7
plt.xlim(-7, 7)
plt.ylim(-7, 7)

# Adicione grade e rótulos
plt.grid(True)
plt.title("Círculo Rasterizado com Decimais")
plt.xlabel("Coordenada X")
plt.ylabel("Coordenada Y")

# Mostre o gráfico
plt.show()
```

Solução 2 - Algoritmo em python

Resultado



Outras opções

Outras Abordagens de Rasterização de círculos:

- **Algoritmo do Ponto Médio (**Midpoint Circle Algorithm)**:** Focado em desempenho (uso apenas de inteiros).

- **Aproximação Poligonal (***Polygon Approximation*)**:** Uso de triângulos para simular a curvatura (padrão em APIs 3D como OpenGL).

- **Algoritmos com Antialiasing (**Xiaolin Wu's Anti-aliased Circle)**:** Focados na suavização das bordas para maior qualidade visual.

Exercícios em aula:

Acessar o arquivo ‘Exercicios_Aula06.pdf’.

Referências e material de apoio

AZEVEDO, Eduardo; CONCI, Aura. Computação gráfica: teoria e prática. Rio de Janeiro: Elsevier, 2003.
COHEN, Marcelo; MANSSOUR, Isabel H. OpenGL: uma abordagem prática e objetiva. São Paulo: Novatec, 2006.
FOLEY, J. et al. Computer graphics: principles and practice. Massachusetts: Addison Wesley, 1997
GOMES, Jonas; VELHO, Luiz. Computação gráfica. Rio de Janeiro: Impa, 1998.
HEARN, Donald; Baker, M. Pauline. Computer grafhics: C version. London: Prentice Hall, 1997.
HETEM JUNIOR, Annibal. Computação gráfica. Rio de Janeiro, RJ: LTC, 2006. 161 p. (Coleção Fundamentos de Informática).
HILL Jr, Francis S. Computer graphics using open GL. New Jersey: Prentice Hall, 2001.
WATT, Alan. 3D computer graphics. Harlow: Addison-Wesley, 2000.
Material Prof. Guilherme Chagas Kurtz, 2023.

Obrigado pelo atenção!!

Email: **andre.flores@ufn.edu.br**

Santa Maria – RS 2026